MPPO-GEM: Reinforcement Learning Approach for Generating Evasive Malware against Static and Dynamic Malware Detectors

Huynh Minh Hien[1,2[0009−0000−8412−0170]], Ngo Trung Hieu[1,2[0009−0002−3150−268X]] , Nguyen Huu Quyen[1,2[0000−0002−0065−9919]], Van-Hau Pham[1,2[0000−0003−3147−3356]], Do Thi Thu Hien[1,2[0000−0001−9387−7909]], and Phan The Duy[1,2[0000−0002−5945−3712]]✉

[1] Information Security Laboratory, University of Information Technology, Ho Chi Minh City, Vietnam
[2] Vietnam National University Ho Chi Minh City, Ho Chi Minh City, Vietnam
{22520415, 22520437}@gm.uit.edu.vn, {quyennh, haupv, hiendtt, ✉duypt}@uit.edu.vn

Abstract. Machine learning (ML)–based malware detectors are widely adopted in both research and practice. However, they remain vulnerable to adversarial attacks in both static and dynamic settings. In the static case, simple semantic-preserving edits (e.g., byte padding, section manipulation, control-flow redividing) can alter model features without breaking executability. In dynamic settings, attackers may evade sandboxing by delaying payload execution, simulating benign API interactions, or employing anti-debug/user-interaction techniques. To analyze these challenges in depth, we propose MPPO-GEM, a MaskablePPO-based framework that jointly applies problem-space primitives for generating adversarial malware, including call-based redividing, section-level modifications, semantic NOP insertion, user-interaction simulation, and anti-debugging bypasses within a constrained, action-masked Reinforcement Learning(RL) environment. The generated adversarial samples are evaluated across multiple platforms (VirusTotal, Kaspersky static/dynamic, MalConv, LightGBM). Empirical results show that our method nearly doubled the evasion rate compared with the original malware while preserving executable integrity.

Keywords: Malware Evasion · Static and Dynamic Analysis · Maskable PPO · Reinforcement Learning.

1 Introduction

Traditional rule- or signature-based antivirus systems (e.g., Kaspersky, VirusTotal) and modern ML–based detectors (e.g., MalConv [12], LightGBM [11]) together form the backbone of current malware defense. Rule-based engines rely on handcrafted signatures, heuristic rules, or behavioral profiles extracted from static and dynamic analysis, but require constant updates and often fail on zero-day samples or obfuscated binaries. To overcome these limitations, the research

community has turned to ML and DL techniques, ranging from byte-level models like MalConv and feature-based LightGBM to graph-based approaches such as MalGraph [10] and MalHAPGNN [17], as well as recurrent and hybrid networks that learn sequential semantics. More recent frameworks, such as FETA [13], further highlight the role of systematic feature engineering in strengthening defenses against anti-static and anti-dynamic evasion. These methods achieve robust performance on large-scale datasets such as EMBER and SOREL-20M. However, despite their effectiveness, both rule-based and ML-based paradigms remain vulnerable to adversarial manipulation: cleverly modified binaries can evade static signatures, delayed or obfuscated behaviors can bypass sandboxing, and subtle feature-preserving perturbations can mislead classifiers. Prior surveys and empirical studies confirm these vulnerabilities in both academic and commercial settings [8, 6, 5, 9], while more recent research explores reinforcement learning as a means to optimize adaptive evasion strategies [15].

To address this limitation, prior work on adversarial malware generation can be broadly divided into three main directions: (i) gradient-based perturbations in feature or byte space [2], (ii) sample generation using data-driven generative models such as GANs and their variants [4, 7], and (iii) RL, where an agent learns a sequential policy of transformations to progressively craft adversarial malware in black-box settings [15, 14, 16]. In the category of generative models, Commey et al. [4] proposed Evolutionary GANs to generate adversarial ransomware variants and evaluated them on VirusTotal and sandbox environments; however, pure GAN-based approaches often struggle to ensure that the produced Portable Executable (PE) files remain valid and executable. In contrast, RL methods can exploit their interaction with the black-box detector—requiring only query responses instead of internal details—to gradually learn effective evasion strategies. For instance, Zhan et al. [15] introduced PSP-Mal, an RL framework with a Shapley prior to prioritize more informative experiences; Rigaki et al. [14] presented MEME, which combines model-based RL with model extraction for improved sample efficiency; and Zhan et al. [16] enhanced exploration in PPO agents through an Intrinsic Curiosity Module (ICM) and GAN-based payload generation to improve diversity. In addition, Ling et al. [9] proposed MalGuise, a black-box query-based attack that uses semantic-preserving transformations with Monte Carlo Tree Search (MCTS) to identify optimal transformation sequences, producing runnable binaries that evade many detectors, although with reduced effectiveness on commercial AV products. Although these approaches achieved strong performance in static detection scenarios, they still exhibit several limitations. For example, Zhan et al.[15] and Rigaki et al.[14] both target static ML detectors and do not address runtime/dynamic evasion, with evaluations primarily on research-grade models. Meanwhile, Zhan et al. [16] and Ling et al.[9] improve RL- or black-box attack strategies but still focus mainly on static research models, with limited validation against dynamic or commercial antivirus systems.

To address these gaps, this study proposes an RL framework for adversarial malware generation that integrates both static and dynamic evasion techniques

within a constrained action space. Specifically, we employ Maskable Proximal Policy Optimization (MaskablePPO), which, to the best of our knowledge, has not been applied to adversarial malware generation before. This approach leverages the action-masking mechanism to ensure that only valid transformations are considered during policy learning, thereby improving convergence and exploration efficiency. The MalConv classifier is used as an oracle for reward computation, guiding the agent toward effective evasion strategies. The adversarial samples produced by our framework are then evaluated against both research-grade models (e.g., MalConv, LightGBM) and commercial environments (e.g., Kaspersky, VirusTotal), demonstrating their real-world evasion capabilities while maintaining executable functionality. The main contributions of this study are threefold:

- We introduce MPPO-GEM, a MaskablePPO-based RL framework with a constrained action space for adversarial malware generation, ensuring both functional integrity and effective evasion across static and dynamic detectors.
- We collect and utilize real-world malware samples from Malware Bazaar and evaluate adversarial binaries generated on multiple platforms, including commercial antivirus systems and research-grade models.
- We present comprehensive comparative results showing that our approach achieves high evasion effectiveness while maintaining executable integrity, outperforming both rule-based products and ML–based detectors.

Furthermore, to provide broader context, Table 1 offers a consolidated overview of representative works on both feature- and problem-space evasion attacks, summarizing their target models, techniques, datasets, and reported success rates.

Table 1: Comparison of proposed method with existing works

Ref.	Target Model	Technique	Dataset	SR
Ling et al.[9]	MalGraph, Magic, MalConv, McAfee, Comodo, Kaspersky,ClamAV, Microsoft Defender ATP (MS-ATP)	Call-based Redividing,	Virustotal, Benign binaries	97-99% (MalGraph, Magic, MalConv), 48.81% (McAfee), 36% (Comodo), 11.29% (Kaspersky), 31.91% (ClamAV), 70.63% (MS-ATP)
Zhan et al.[15]	Ember, Sorel-20M	RL with Shapley prior	VirusTotal (Malware), PortableApps (Benign)	72-76 %
Rigaki et al.[14]	Ember, Sorel-LGB, Sorel-FFNN, Microsoft Defender AV	MEME (Model-based RL + Model Extraction)	Ember 2018, Sorel-20M, Benign binaries	32-73 %
Commey et al.[4]	VirusTotal, Kaspersky sandbox, Cuckoo sandbox	EGAN (Evolutional Generative Adversarial Network)	VirusTotal	100% (Kaspersky), 0% (Cuckoo)
Zhan et al.[16]	EMBER, FireEye, MalConv	RL (PPO) with ICM, GAN (DP-GAN)	VirusTotal (Malware), PortableApps (Benign)	63.3-85.5%
Aryal et al.[2]	MalConv, MalConv2	Intra-Section Code Cave Injection with Gradient Descent and FGSM	Virustotal (Malware), Kaggle (Benign)	92.31-97.93%
Gibert et al.[7]	EMBER, Virustotal	Conditional Wasserstein GAN	BODMAS	
Qiao et al.[12]	MalConv	Prototype-based Adversarial Malware Generation	Vxheaven, Benign Binaries	68.5-92.0%
MPPO-GEM (Our work)	Virustotal, Kaspersky, MalConv, LightGBM	Call-based Redividing, MaskablePPO	Malware Bazaar	26.96-78.93% (static), 20.35-88.74% (dynamic)

The remainder of this paper is organized as follows. **Section 2** describes in detail the proposed MPPO-GEM framework. **Section 3** presents the experi-

mental setup and results. Finally, **Section 4** concludes the study and outlines directions for future research.

2 Proposed Method

This section presents the proposed system through three components: an overview of the architecture and preprocessing pipeline, an MDP formulation defining states, actions, and rewards for executable transformations, and the MaskablePPO training method, which enables stable learning by masking invalid actions (see Figure 1 for training and testing phases).

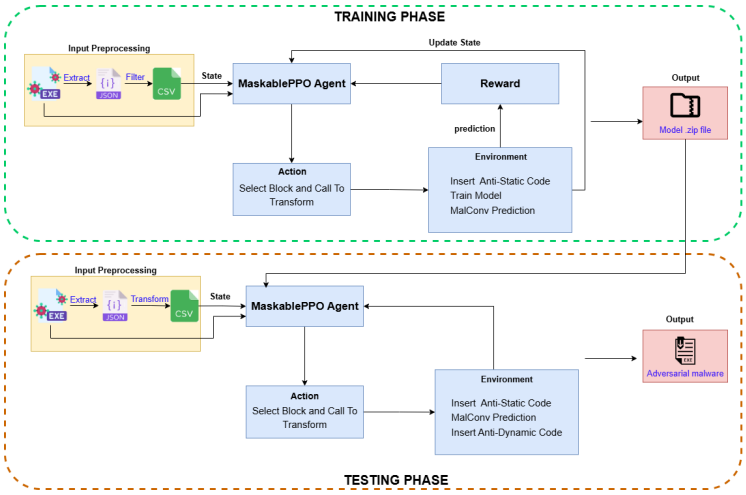

Fig. 1: MPPO-GEM Overview

2.1 Overview

As illustrated in Figure 1, the method trains a single Maskable PPO agent in a two-phase (training and testing). PE files are preprocessed into a state (CFG into feature); during training, the agent selects suitable block and call for transformations, the environment applies anti-static edits, rebuilds the binary, and queries a detector MalConv to produce a reward that favors lower detection. The resulting policy is used in testing to generate adversarial malware with anti-static and anti-dynamic code, aiming to reduce the detection probability of static and dynamic detectors without altering program behavior.

2.2 Markov Decision Process Model

In this project, we model the malware evasion process as a Markov Decision Process (MDP),

$$\mathcal{E} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, R, \gamma \rangle,$$

where $\mathcal{S}$ is the state space, $\mathcal{A}$ the action space, $\mathcal{P}(s' \mid s, a)$ the transition distribution, R the reward function, and $\gamma \in [0, 1]$ the discount factor.

At each step, the agent observes a normalized vector $o_t \in [0, 1]^5$ describing the current code block and call instruction, and selects a valid action $a_t \in \mathcal{A}(o_t)$ to modify the binary. The reward is based on the malware detector's predicted probability p_t, with $r_t = 1 - p_t$, encouraging transformations that reduce detection. Episodes terminate either when a maximum number of steps is reached or when the candidate successfully evades detection.

State. $\mathcal{S}$ The internal state of the environment at step t is defined as the number of actions taken since the beginning of the episode:

$$s_t \in \mathbb{S}_{\geq 0}, \quad s_0 = 0.$$

Note: s_t is an internal state used to track the episode progression (e.g., to stop after a maximum number of steps). It is not directly provided to the agent for decision-making.

Action. $\mathcal{A}$ The action space is denoted by $\mathcal{A}$. At step t, the feasible actions are constrained by an action mask m_t:

$$a_t \in \mathcal{A}(o_t) \subseteq \mathcal{A},$$

where each action a_t corresponds to a concrete code transformation (e.g., call redirection, semantic NOP insertion). Insertion points are chosen by estimating the required payload size and locating suitable code caves; if none are available, a small aligned executable section is allocated — only locations that pass basic validity checks are exposed to the agent via the action mask.

In addition, the agent receives an *observation* o_t as a continuous vector encoding structural features of the current code block and its call instruction:

$$o_t = \left[o_{\text{start}},\, o_{\text{end}},\, o_{\text{call}},\, o_{\text{target}},\, o_{\text{execlen}}\right],$$

where each component is:

- o_{start} — the start virtual address of the current code block, normalized by the file size.
- o_{end} — the end virtual address of the current code block, normalized by the file size.
- o_{call} — the virtual address of the `call` instruction within the current block, normalized by the file size.
- o_{target} — the virtual address of the call target, normalized by the file size.
- o_{execlen} — the length of executable bytes in the current PE file, normalized by the maximum allowed length (e.g., 524,288 bytes).

Reward. $\boldsymbol{R}$ The reward is computed on the basis of the malware detection probability predicted by the detector for the candidate sample. Let

$$p_t \in [0, 1]$$

denote the predicted probability that the modified sample is detected as malware. The scalar reward is defined as:

$$r_t = 1 - p_t,$$

so that lower detection probability (more evasive malware) yields higher reward. An episode is considered successful if

$$p_t \leq p_{\text{stop}} \quad \text{or equivalently} \quad r_t \geq r_{\text{stop}} := 1 - p_{\text{stop}}.$$

Environment. $\boldsymbol{\mathcal{E}}$ The environment consists of the malware detector (MalConv) and the transformation interface applied to the binary. At each step, the agent selects an action $a_t \in \mathcal{A}$ to modify the executable. The modified binary is then evaluated by the MalConv detector, which produces: 1) the observation o_{t+1}, encoding the structural context of the relevant code block and call instruction, and 2) the reward $r_t = 1 - p_t$, where $p_t \in [0, 1]$ is the detector's predicted probability that the candidate is malware.

The environment may introduce controlled stochasticity (e.g., randomized semantic NOP insertion or code-cave selection) to increase sample diversity while preserving executable functionality. An episode terminates when the maximum number of steps is reached or when the candidate successfully evades detection ($p_t \leq p_{\text{stop}}$).

2.3 MPPO-GEM Method

Training Phase: In this phase, an initial malware sample x_0 is provided to the system and preprocessed to extract a feature representation, yielding the initial state s_0. A MaskablePPO agent parameterized by policy π_θ receives s_0 and, together with an action mask m_t, selects a valid action a_t. Each chosen action corresponds to a transformation applied to the current sample x, producing a modified variant x_{mod}. Moreover, to enable x_{mod} to evade dynamic detectors and sandboxes, we added anti-dynamic techniques such as anti-sandbox, anti-debugging, timing checks, control-flow obfuscation, and user-interaction techniques. From x_{mod} the representative bytes b_t are extracted and queried against the detector, which returns a detection probability p_t. The immediate reward r_t is defined as a function of p_t (for example $r_t = 1 - p_t$), thus encouraging the agent to learn transformations that reduce the likelihood of detection.

If in any timestep $p_t \leq p_{\text{stop}}$, the modified variant x_{mod} is accepted and committed as the new current sample x. This condition may also trigger early termination of the episode to conserve computation. After at most N episodes with up to T steps per episode, the training procedure returns the best-found

Algorithm 1 MPPO-GEM Method

```
Require: x_0                                          ▷ initial malware sample
Require: detector                                     ▷ black-box scoring function (MalConv)
Require: policy π_θ                                   ▷ Maskable PPO policy parameterized by θ
Require: S                                            ▷ set of candidate action indices (from CSV)
Require: T                                            ▷ max steps per episode (inner loop)
Require: N                                            ▷ max episodes / outer iterations
Require: p_stop                                       ▷ detector score threshold for success
Ensure: x*                                            ▷ adversarial malware sample (best found)
x ← x_0                                               ▷ committed sample
for n ← 1 to N do
    s_0 ← extract_features(x)
    for t ← 0 to T − 1 do
        m_t ← get_action_mask(x)
        a_t ← π_θ.predict(s_t, action_mask = m_t)
        (success, x_mod) ← apply_transformation(x, a_t)
        if not success then
            s_{t+1} ← extract_features(x)             ▷ no change
            continue
        end if
        b_t ← get_representative_bytes(x_mod)
        p_t ← query_detector(detector, b_t)           ▷ p_t ∈ [0, 1]
        r_t ← 1 − p_t
        s_{t+1} ← extract_features(x_mod)
        if p_t ≤ p_stop then
            x ← x_mod                                 ▷ commit success
            break                                     ▷ early success in episode n
        end if
        x ← x_mod                                     ▷ commit latest modification
    end for
    b_final ← get_representative_bytes(x)
    p_final ← query_detector(detector, b_final)
    if p_final ≤ p_stop then
        return x* ← x
    end if
end for
return x* ← x
```

adversarial sample x^* and the optimized policy π_θ. The trained policy persists for subsequent evaluation in the testing phase.

Testing Phase: In this phase, an input sample (x_0) is preprocessed to obtain the initial state s_0. The MaskablePPO agent applies the trained policy π_θ in conjunction with the action mask m_t to select a sequence of actions a_t. Each action is executed by the environment on the current sample x, resulting in modified variants x_{mod}. Like in the **Training Phase**, x_{mod} is also equipped with anti-dynamic techniques. After each transformation, the detector provides a detection probability p_t. When the condition $p_t \leq p_{\text{stop}}$ is met, the current variant is committed and regarded as a successful evasive example.

Crucially, during testing, the policy π_θ is *not* updated; it is used deterministically (or stochastically, if desired) to transform the input according to the strategy learned during training. The final output of the testing phase is an adversarial sample x^*: a modified malware instance that successfully evades detection under the evaluated detector.

Note: Algorithm 1 is combined both phases, training phase and testing phase.

3 Experiments

In this section, we describe the dataset collection and the experimental procedures used to evaluate our proposed framework.

3.1 Experiment Settings

All experiments were conducted on Kaggle (Ubuntu 22.04 LTS, NVIDIA T4 GPU, 16 GB RAM) using Python 3.11.3 with PyTorch 2.0 and Stable-Baselines3. PE features were extracted with IDA Pro 6.6, and semantic-preserving transformations were applied via LIEF 0.17.0.

3.2 Evaluation Metrics

We evaluate detector performance using four standard classification metrics and one adversarial robustness metric.

Classification metrics:

- **Accuracy (Acc)**: the fraction of correctly classified samples (both benign and malicious).
- **Precision (Pre)**: the fraction of predicted-malicious samples that are truly malicious (measures false positive control).
- **Recall (Rec)**: the fraction of actual-malicious samples that are correctly detected (true positive rate).
- **F1-Score (F1)**: the harmonic mean of precision and recall, balancing false positives and false negatives.

Evasion Rate: Given a detector score $s(x)$ and threshold τ, a malware sample x is considered to evade detection if $s(x) < \tau$. We compute the evasion rate on a held-out adversarial test set $\mathcal{M}_{\text{adv}}$ as:

$$\text{EvasionRate}(\mathcal{M}_{\text{adv}}) = \frac{|\{x \in \mathcal{M}_{\text{adv}} : s(x) < \tau\}|}{|\mathcal{M}_{\text{adv}}|} \times 100\%.$$

3.3 Dataset

For experimental evaluation, we constructed two datasets from publicly available sources. A total of 10,277 malware samples were collected from Malware Bazaar [1], and 5,204 benign Windows executables were obtained from the GitHub Benign-Net [3] project. Among them, 5,309 malware samples together with all 5,204 benign files were used to train the MalConv and LightGBM detectors. The remaining 4,968 malware samples from Malware Bazaar were reserved for RL, where 4,000 were employed to train the MaskablePPO agent, and 968 were kept as a hold-out test set for adversarial evaluation.

3.4 Pre-processing and Feature Extraction

Prior to adversarial generation, all PE samples underwent an automated preprocessing pipeline to extract structural and control-flow information used by the RL agent. Static analysis was performed with industry-standard disassembly tooling (IDA Pro 6.6) and accompanying automation scripts to produce a per-file, machine-readable representation. For each binary we recorded, at a high level, the following items:

- **Code block boundaries:** normalized start and end offsets corresponding to $(o_{\text{start}}, o_{\text{end}})$.
- **Call instruction context:** the relative call address and its target, corresponding to $(o_{\text{call}}, o_{\text{target}})$.
- **Executable length:** the global file size used for normalization, o_{execlen}.

These features form the observation vector $o_t = [o_{\text{start}}, o_{\text{end}}, o_{\text{call}}, o_{\text{target}}, o_{\text{execlen}}] \in [0,1]^5$ provided to the RL agent at each step. All representations were serialized to JSON (UTF-8, schema v1) and exported as CSV with header: `pe, block_start, block_end, call_address, call_target, execlen`.

All preprocessing steps were executed deterministically and logged (tool versions, scripts). Files that failed validation or parsing were excluded and enumerated in a manifest file. The resulting CSV and JSON artifacts were used as abstract feature inputs to the RL training and evaluation pipeline; raw binaries and extraction logs were archived to enable auditability and reproducibility.

3.5 Model Configurations and Hyperparameters

We employ three models in our study: MaskablePPO to generate adversarial malware, and LightGBM together with MalConv as detectors to evaluate evasion effectiveness.

MaskablePPO. To train the agent, we adopted MaskablePPO with a constrained action space and an `MlpPolicy` (two hidden layers of 128 units for the actor and 512 units for the critic). Training was performed with a learning rate of 2×10^{-5}, rollout length of 256 steps, batch size of 128, and 12 epochs per update. We set $\gamma = 0.99$, $\lambda = 0.98$, clip range $= 0.1$, entropy coefficient $= 0.01$, and value function coefficient $= 0.25$, with normalized advantages to improve stability.

LightGBM. The LightGBM-based detector follows the original configuration in [11], using gradient boosting decision trees trained on the same dataset. We adopt the parameter settings reported in the reference paper.

MalConv. The MalConv detector is implemented according to the description in [12], directly consuming raw PE byte sequences through convolutional layers with a gating mechanism. We keep the configuration identical to the original work.

3.6 Experimental Scenario

To establish evaluation baselines, MalConv and LightGBM were trained on 5,309 malware and 5,204 benign executables with a 70:30 split, ensuring strict hash-based separation. Their **Acc**, **Pre**, **Rec**, and **F1** on the held-out test set were recorded before freezing the models as fixed oracles.

Separately, 4,968 unseen malware samples were reserved for MaskablePPO. Of these, 4,000 were used to train the RL agent with semantic-preserving transformations and action masking, while 968 formed a hold-out test set. This disjoint split enabled a clean assessment of evasion performance and generalization.

3.7 Performance Benchmarking with Existing Approaches

Table 2 reports MalConv and LightGBM's baseline performance metrics on the held-out test subset, showing high **Acc** and balanced **Pre**–**Rec** across both detectors.

Table 2: Performance Evaluation of MalConv and LightGBM

Detector	**Acc**	**Pre**	**Rec**	**F1**
MalConv	0.9534	0.9534	0.9534	0.9534
LightGBM	0.9360	0.9430	0.9360	0.9357

As reported in Table 2, both MalConv and LightGBM achieve strong baseline performance on the held-out test split, with balanced precision and recall. These results confirm that the detectors provide a reliable reference point for subsequent adversarial evaluations.

We then evaluated our MPPO-GEM–generated adversarial malware against multiple detectors, including VirusTotal (static and dynamic modes), Kaspersky (static and dynamic), MalConv, and LightGBM. Table 3 summarizes detection rates before and after transformation. Figure 2 visualizes the reduction in the number of VirusTotal engines that flag a sample as malicious between original and adversarial variants. Note that MalConv's adversarial detection rate is shown for reference only, since MalConv predictions were used as the reward signal during RL training.

Table 3: Evasion Rates of Original vs. Adversarial Malware

Sample	**VirusTotal (dynamic)**	**Kaspersky (static)**	**Kaspersky (dynamic)**	**MalConv**	**LightGBM**
Original malware	54.44%	10.02%	10.74%	22%	35.12%
Adversarial malware	88.74%	26.96%	20.35%	65.08%	78.93%

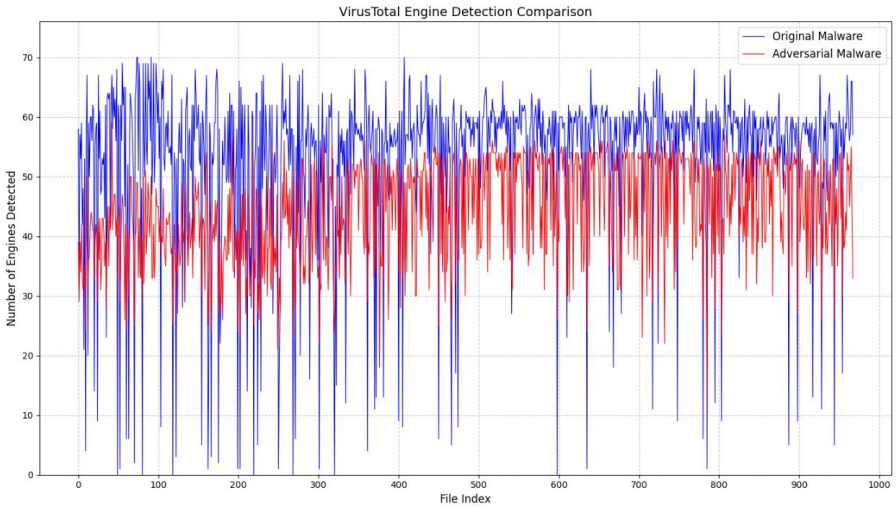

Fig. 2: Reduction in VirusTotal engine detections between original and adversarial samples.

Table 3 and Figure 2 indicate that our MPPO-GEM–generated adversarial samples substantially increase the evasion rate against several detectors (notably LightGBM and MalConv). At the same time, because VirusTotal static results represent an aggregation of many independent AV engines (each with its own signatures and heuristics), neither the original nor the adversarial samples can be expected to completely evade VirusTotal static detection as a whole. Nevertheless, the per-engine analysis in Figure 2 reveals a clear reduction in the number of engines that flag the adversarial variants, corroborating the effectiveness of our problem-space semantic transformations. Importantly, dynamic evaluation also shows consistent improvements: evasion rose from 54.44% to 88.74% on VirusTotal dynamic and from 10.74% to 20.35% on Kaspersky dynamic, confirming that the proposed framework remains effective even under sandbox-based behavioral analysis.

4 Conclusion

This study investigated the generation of adversarial malware using MPPO-GEM, a MaskablePPO-based RL framework, and evaluated its effectiveness across rule-based and ML–based detectors. Empirical results highlight that our method achieved an evasion rate of 65.08% against MalConv—the lowest among ML-based detectors—and 20.35% against Kaspersky dynamic analysis—the lowest among rule-based detectors. These results demonstrate that the proposed framework is capable of achieving strong evasion performance while preserving executable functionality, suggesting its potential for practical application. Future

work will focus on improving robustness by extending the range of semantic-preserving transformations, refining the reward design to better capture evasion objectives, and enabling transformations across multiple basic blocks rather than a single block. In addition, large-scale validation will be conducted in more diverse datasets and security products to ensure generalizable effectiveness in real-world adversarial settings.

5 Acknowledgements

This research is funded by University of Information Technology - Vietnam National University HoChiMinh City under grant number of D1-2025-49.

References

1. Abuse.ch: MalwareBazaar Dataset. https://datalake.abuse.ch/malware-bazaar/daily/ (2025)
2. Aryal, K., Gupta, M., Abdelsalam, M., Saleh, M.: Intra-section code cave injection for adversarial evasion attacks on windows pe malware file. arXiv preprint arXiv:2403.06428 (2024)
3. bormaa: Benign .net files (2023), https://github.com/bormaa/Benign-NET, a repository containing benign .NET executable files collected from CNET, Softonic, SourceForge, and other sources, used for malware detection research
4. Commey, D., Appiah, B., Frimpong, B.K., Osei, I., Hammond, E.N., Crosby, G.V.: Egan: Evolutional gan for ransomware evasion. In: 2023 IEEE 48th Conference on Local Computer Networks (LCN). pp. 1–9. IEEE (October 2023)
5. Demetrio, L., Biggio, B., Roli, F.: Practical attacks on machine learning: A case study on adversarial windows malware. IEEE Security & Privacy **20**(5), 77–85 (2022)
6. Geng, J., Wang, J., Fang, Z., Zhou, Y., Wu, D., Ge, W.: A survey of strategy-driven evasion methods for pe malware: Transformation, concealment, and attack. Computers & Security **137**, 103595 (2024)
7. Gibert, D., Planes, J., Le, Q., Zizzo, G.: A wolf in sheep's clothing: Query-free evasion attacks against machine learning-based malware detectors with generative adversarial networks. In: 2023 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW). pp. 415–426. IEEE (2023)
8. Ling, X., Wu, L., Zhang, J., Qu, Z., Deng, W., Chen, X., Wu, Y.: Adversarial attacks against windows pe malware detection: A survey of the state-of-the-art. Computers & Security **128**, 103134 (2023)
9. Ling, X., Wu, Z., Wang, B., Deng, W., Wu, J., Ji, S., Wu, Y.: A wolf in sheep's clothing: practical black-box adversarial attacks for evading learning-based windows malware detection in the wild. In: 33rd USENIX Security Symposium (USENIX Security 24). pp. 7393–7410. USENIX Association (2024)
10. Ling, Y., Fang, Y., Zhang, L., Li, K., Zhou, J.: Malgraph: Hierarchical graph neural networks for robust windows malware detection. In: Proceedings of IEEE INFOCOM. pp. 2589–2598. IEEE (2022). https://doi.org/10.1109/INFOCOM48880.2022.9796737

11. Nguyen, A.T., Raff, E., Nicholas, C., Holt, J.: Leveraging uncertainty for improved static malware detection under extreme false positive constraints. arXiv preprint arXiv:2108.04081 (2021)
12. Qiao, Y., Zhang, W., Tian, Z., Yang, L.T., Liu, Y., Alazab, M.: Adversarial malware sample generation method based on the prototype of deep learning detector. Computers & Security **119**, 102762 (2022)
13. Rabadi, D., Loo, J.Y., Narayanan, A., Wang, Y., Teo, S.G., Truong-Huu, T.: Feta: A systematic and efficient approach for feature engineering on anti-static and anti-dynamic malware analysis. Journal of Information Security and Applications (2025)
14. Rigaki, M., Garcia, S.: The power of meme: Adversarial malware creation with model-based reinforcement learning. In: European Symposium on Research in Computer Security. pp. 44–64. Springer Nature Switzerland, Cham (September 2023)
15. Zhan, D., Bai, W., Liu, X., Hu, Y., Zhang, L., Guo, S., Pan, Z.: Psp-mal: Evading malware detection via prioritized experience-based reinforcement learning with shapley prior. In: Proceedings of the 39th Annual Computer Security Applications Conference. pp. 580–593. ACM (Dec 2023)
16. Zhan, D., Zhang, Y., Zhu, L., Chen, J., Xia, S., Guo, S., Pan, Z.: Enhancing reinforcement learning based adversarial malware generation to evade static detection. Alexandria Engineering Journal **98**, 32–43 (2024)
17. Zhao, Z., Chen, Y., Xu, H., Liu, Q., Wang, J.: Malhapgnn: An enhanced call graph-based malware detection framework using hierarchical attention pooling graph neural network. IEEE Transactions on Information Forensics and Security **18**, 1234–1247 (2023). https://doi.org/10.1109/TIFS.2023.3245678